

Everything you've typed so far

Every command, meta-command, and function actually used hands-on across this project — grouped by tool, in the order you met them, not alphabetized — plus a project-wide file map, and a running list of Claude's own working commands. Appended at the end of each build, not written once and frozen.

COVERS

Build 1 (agent) through Build 7 (verifier/critic agent) — all complete

ENVIRONMENT

Windows 11 · PowerShell · Docker Desktop · GitHub Actions

DATABASES

pg_local (appdb)

Project Files

Docker

psql

PowerShell

Git

GitHub CLI

psycopg

psycopg.sql

pandas

pandera

pytest

pgvector & embeddings

BeautifulSoup

Python stdlib

Streamlit

Anthropic SDK

Claude's Commands

Project Files

WHAT'S ACTUALLY IN THIS FOLDER

A map of the project root as of the 2026-08-05 repo reorganization — what each file is for, not what commands it contains (that's every section below). Excludes auto-generated folders (`.venv/`, `__pycache__/`, `.git/`).

Reorg, 2026-08-05: a full project audit (prompted by a real CI failure — see the Claude's Commands section for what it found) turned up a flat ~50-file root that had never been reorganized as the project grew across seven builds, plus two stray files and a badly out-of-date `.env.example`. Everything below reflects the result: first-party Python in `src/` (kept flat — the import graph between `agent.py/tools.py/verify_answer.py`/the eval harnesses rules out further splitting without adding `sys.path` config), SQL migrations in `migrations/`, tests in `tests/` (`pytest.ini` gained `pythonpath = src`), diagrams/briefs/cheat sheet in `docs/` (renamed from `PROJECT_DIAGRAMS/`), and the two vestigial SQL Server lab files in `legacy/`. `db_test_py.py` and an empty VS Code scratch file were deleted outright — same disposition Build 5 gave `api_test.py`.

SQL migrations — `migrations/`, run in order, numbered for reproducibility

<code>migrations/001_schema.sql</code>
Original appdb schema — customers/orders, the seed data every later build extends.
<code>migrations/002_seed.sql</code>
Idempotent seed data for customers/orders.
<code>migrations/003_readonly_role.sql</code>
appdb_reader — the read-only role Build 1's agent runs as.
<code>migrations/004_raw_schema.sql</code>
Build 2 Phase 1 — raw.allocations, all-TEXT ingestion target.
<code>migrations/005_clean_schema.sql</code>
Build 2 Phase 2 — clean.allocations, real types plus the computed duration_seconds column.
<code>migrations/006_agent_scratch_role.sql</code>
Build 2.5 Phase 1 — appdb_agent_writer, the agent's write-scoped role.
<code>migrations/007_agent_scratch_schema.sql</code>
Build 2.5 Phase 1 — the agent_scratch schema and its grants.
<code>migrations/008_add_summary_embedding.sql</code>
Build 3 Phase 1 — adds clean.allocations.summary_embedding vector(384).

migrations/009_add_summary_embedding_hnsw_index.sql

Build 3 follow-up — HNSW index on summary_embedding for indexed similarity search.

migrations/010_docs_chunks_schema.sql

Build 3.5 Phase 1 — the docs.chunks table (vector(384)) for the documentation RAG corpus.

migrations/011_metrics_history_schema.sql

Build 4 follow-up — observability.metrics_history, a tracked time series over metrics_rollup.py's runs.

migrations/012_chat_rounds_schema.sql

Build 6 Phase 3.5 — agent_scratch.chat_rounds, persistent shared chat history surviving a real container restart. Missing from CI's hardcoded migration list from the day it was written until the 2026-08-05 audit caught it — see Claude's Commands.

migrations/013_allocation_items_schema.sql

2026-08-06 — clean.allocation_items, one row per checked-out item (item_name, category, is_accessory, is_returned, tag), replacing query-time parsing of summary's delimited text. Populated by src/build_allocation_items.py, not SQL alone — see the 2026-08-06 token-burn incident audit for why this exists.

Python — the agent itself (src/)

src/agent.py

The hand-built tool-use loop, wrapped in a callable ask_agent() since Build 4 — calls the Anthropic API, dispatches tool calls, enforces MAX_TOOL_TURNS, tags every logged call with a question_id. Build 7 — every successful answer with real tool evidence now also runs through verify_answer() before returning, wrapped in try/except so a verifier failure can never block the real answer. 2026-08-06 — tracks per-turn Anthropic token usage (printed to console, logged via log_final_answer); sys.stdout.reconfigure(encoding="utf-8") added after a real UnicodeEncodeError crash (Windows' default cp1252 console encoding can't represent characters Claude's own output routinely uses, e.g. →). SYSTEM_PROMPT went through 5 iterations the same day chasing a 177,090-token/wrong-answer question — the first four tried to teach the model to parse summary's delimited text correctly (aggregate queries, accessory-keyword traps, a mandatory full-list-first rule); each fixed one symptom while introducing another. The prompt actually got *shorter* once clean.allocation_items (migration 013) existed to point at instead — full account in the 2026-08-06 token-burn incident audit.

src/tools.py

Every tool the agent can call: run_sql_query, list_tables, describe_table, save_dataframe, search_summaries, search_docs. run_sql_query has two independent safety caps: MAX_SQL_RESULT_ROWS = 200 (row count) and, added 2026-08-06, MAX_FIELD_CHARS = 500 (single-value size) — a query can blow the token budget past either cap alone even while respecting the other, confirmed live when a 20-row GROUP BY summary result (well under the row cap) still cost 70,007 tokens for one question because individual summary fields ran 1,500+ characters; the same fix dropped a SELECT * result from 55,504 to 12,051 characters by capping the summary_embedding vector column too.

Python — observability & eval harness (Build 4, src/)

src/logging_utils.py

Structured JSON Lines tool-call logging — one line per call: timestamp, question_id, turn, tool name, input, output, error, latency. Build 7 — get_tool_calls() gained an include_output flag (default False, existing callers unaffected); new get_verification()/log_verification(), a third distinct log-entry shape keyed on supported so it can never collide with get_final_answer()'s "answer" filter.

src/eval_cases.py / src/eval_cases_round2.py

Fixed sets of test questions with an expected tool and expected answer keywords — run end-to-end against the live agent, not tested in isolation. Two cases fixed 2026-08-06 after real observed flakiness, not guessed: the "agent_scratch two roles" question reworded to directly ask for both role names (an open-ended "why" question let the model paraphrase around a literal identifier some runs); the stale "total revenue from orders" question (customers/orders left the agent's scope in Build 6) reframed around the grounded, correct behavior — redirecting to `clean.allocations` — instead of expecting a fabricated \$ figure, with `expected_tool` now a list since `list_tables/search_docs/run_sql_query` are all legitimate paths to that same correct answer.

src/run_eval.py

Drives every case through `ask_agent()`, checks the tool-call log and answer text, reports pass/fail per question. Accepts an optional case-module argument. Real bug found 2026-08-05, the first time this ran in CI after weeks of nothing reaching origin/main: `tools_called` assumed every new log line was a tool-call entry, but `log_final_answer/log_verification` entries (Build 6/7) share the same file and lack `tool_name` — fixed with the same "tool_name" in `entry guard logging_utils.py` already uses. 2026-08-06 — `expected_tool` can now be a list, not just one string, for cases where several different tools are all legitimate ways to reach the same correct conclusion.

src/metrics_rollup.py

Plain-text report over the tool-call log — tool-usage frequency, average turns per question (grouped by `question_id`), error rate.

Python — live deployment with CI (Build 5, src/)**src/synthesize_dataset.py**

Builds `data/ALLOCATION-synthesized.csv` from the real export — real dates/duration/renewal/resource counts kept, department/email/summary synthesized. Imports the real department-code mapping from a git-ignored local file, never hardcodes it.

department_mapping.local.py

Git-ignored, repo root — the real department-code mapping `synthesize_dataset.py` actually runs from. Never committed; see `DEPARTMENT_MAPPING.local.md` for the human-readable copy.

pytest.ini

Repo root — `python_files = test_*.py` scopes discovery to this project's actual test-file convention, after a stray non-test script (`api_test.py`) was found being silently collected by `pytest`'s broader default. Gained `testpaths = tests / pythonpath = src` in the 2026-08-05 reorg.

Python — front end for the agent (Build 6, src/ + tests/)**src/app.py**

The Streamlit entrypoint — chat UI wrapping `ask_agent()`, hybrid tiered history threading via `session_state.rounds`, a persisted feed (Phase 3.5) loaded from `agent_scratch.chat_rounds` on first load, and an FAQ-style render: only the newest round shows expanded, older rounds collapse into `st.expander(label=the question)`. Build 7 — `render_verification_badge()` shows a green `st.success()`/red `st.error()` status right after each answer (nothing rendered when there's no verification entry at all), using Streamlit's own safe text escaping rather than raw HTML since the verdict's reason text is LLM-generated.

tests/test_agent_history.py

3 tests for `flatten_history()` — the pure function behind hybrid tiered conversation memory.

tests/test_chat_rounds.py

3 tests — save/load round-trip, discovery exclusion, and `MAX_CHAT_ROUNDS`'s exact boundary via monkeypatch.

Python — verifier/critic agent (Build 7, src/ + tests/)

src/verify_answer.py

The verifier itself — a second, narrower agent judging whether a proposed answer is supported by the tool evidence gathered for it. Forces `tool_choice` so the verdict is always a structured `{supported, reason}`, never prose. Importable (`agent.py` uses it directly) and CLI-runnable standalone against any real logged `question_id`, with an optional `override-answer` argument to prove the deliberately-bad case. 2026-08-06 — `VERIFIER_SYSTEM_PROMPT` taught to check whether a per-category SQL condition is a true subset of a stricter total's condition, not just compare the resulting numbers, after a real false positive; a real false *negative* found later the same day (a correct accessory-exclusion judgment flagged as unsupported) resolved on its own once the underlying data was normalized — no further verifier changes needed.

src/verify_eval_cases.py

Hand-constructed evidence + answer pairs (not read from the live log, for CI-portability) proving the verifier catches a bad answer, not just that it never complains — a grounded case, an outright contradiction, a fabricated number, plus two look-alike cases added 2026-08-06 after a real false-negative: per-category SQL counts broader than the total they're compared against (should fail) vs. genuine overlapping categories that legitimately sum past a total (should pass) — same surface shape, opposite correct verdicts.

src/run_verify_eval.py

Mirrors `run_eval.py`'s shape — runs every case through `verify_answer()`, full per-case report, non-zero exit on any mismatch. Never wired into CI until the 2026-08-05 audit — see Claude's Commands.

tests/test_verification.py

2 tests for `get_verification()`'s found/not-found paths — temp log file + monkeypatch, no API cost for the pure-logic half of Build 7.

Python — data pipeline (Build 2, src/)

src/ingest.py

Loads the source CSV into `raw.allocations`, one `executemany` call.

src/profile_raw_data.py

Pandas exploration of `raw.allocations` that informed `005_clean_schema.sql`'s design. Renamed from `profile.py` — it silently shadowed Python's `stdlib` `profile` module.

src/transform.py

Raw → clean transform: type coercion, blank→NULL handling, populates `clean.allocations`.

src/build_allocation_items.py

2026-08-06 — parses `clean.allocations.summary` (pipe-delimited, "Returned: " prefix stripped) into `clean.allocation_items`, one row per item, via a closed `ITEM_CLASSIFICATION` lookup for all 55 distinct item names found by a full audit of the data — fails loudly on any unclassified name rather than silently miscategorizing it. Full rebuild (`TRUNCATE + reload`) each run, not incremental — the source is a closed, fully-read set of strings, not a growing feed.

Python — semantic search (Build 3, src/)

src/embed_summaries.py

Idempotent backfill — embeds every summary row still missing a `summary_embedding`.

`src/search_summaries.py`

Standalone script proving raw cosine-distance search works, before any agent wiring.

`src/calibrate_threshold.py`

Statistical calibration of the 0.63 relevance-distance cutoff, from 40 curated in-/out-of-domain queries.

Python — documentation RAG (Build 3.5, `src/`)

`src/extract_doc_chunks.py`

Parses this project's own docs into embeddable chunks — four extraction shapes across two design eras, plus narrative prose. `SOURCE_FILES` auto-discovers every handoff brief via `glob` (Build 6 fix) rather than a hand-maintained list, after that list twice silently fell behind a newly-written brief. Glob target updated to `docs/` in the 2026-08-05 reorg (was `PROJECT_DIAGRAMS/`).

`src/embed_chunks.py`

Idempotent backfill for `docs.chunks` — same `WHERE embedding IS NULL` pattern as `embed_summaries.py`.

`src/search_chunks.py`

Standalone script proving raw similarity search over the docs corpus works, before any agent wiring.

Tests (`tests/`)

`tests/test_data_quality.py`

Build 2 — `pandera` schema checks against `clean.allocations`.

`tests/test_agent_scratch_boundary.py`

Build 2.5 — role-boundary tests plus the `MAX_SCRATCH_TABLES` cap test.

`tests/test_semantic_search.py`

Build 3 — embedding coverage, relevant/irrelevant query behavior, HNSW index existence.

`tests/test_docs_search.py`

Build 3.5 — same shape as `test_semantic_search.py`, applied to the documentation corpus.

Configuration & infrastructure (repo root)

`README.md`

Added in the 2026-08-05 reorg — the repo's actual entry point (what this is, quickstart, folder layout, trust boundary). Didn't exist before.

`docker-compose.yml`

Defines `pg_local`, `mssql_local`, the agent service, and (Build 6) `streamlit_app` — same image as agent, just a different `command`:. Postgres image is `pgvector/pgvector:pg17`. `streamlit_app`'s command updated to `src/app.py` in the reorg.

.streamlit/config.toml 2026-08-06 — fileWatcherType = "none". Originally suspected as the cause of a 36.5s latency spike (its module-tree walk produces harmless torchvision ModuleNotFoundError noise timestamped inside the slow window); disproven by testing a non-Streamlit process in the same container, which was equally slow. Kept anyway — it's a real, separate fix for the documented log-noise issue, just not the latency one (that turned out to be the missing model-cache fix, see the Dockerfile row above).
Dockerfile Builds one shared image used by both the agent and streamlit_app services — differentiated only by each service's command: override, not two separate Dockerfiles. Build 7 — COPY line fixed to include verify_answer.py. Reorg — COPY/CMD updated to src/ paths. 2026-08-06 — gained a RUN step that downloads the all-MiniLM-L6-v2 embedding model at build time, after a fresh/recreated container's first embedding call was measured at 36.5s (a full network re-download) vs. 2.4s once cached; baking it into the image removes the runtime network dependency entirely rather than just caching it after the fact. Also copies in .streamlit/config.toml.
requirements.txt Pinned Python dependencies — anthropic, beautifulsoup4, psycopy, pgvector, sentence-transformers, pandas, pandera, pytest, python-dotenv, streamlit (Build 6).
.env Real local secrets (DB passwords, API key) — gitignored, never committed.
.env.example Template shape of .env. Rewritten in the 2026-08-05 audit — the prior version documented 4 of the 10 vars the code actually requires (missing ANTHROPIC_API_KEY and both reader/writer role credentials entirely), meaning a fresh clone following it hit an immediate KeyError.
.gitignore Excludes secrets (.env, CREDENTIALS.local.md, DEPARTMENT_MAPPING.local.md, department_mapping.local.py), OS/editor cruft, __pycache__/, logs/dumps. Personal working-style reference path updated to docs/Working-Style_Prompt/ in the reorg.
.github/workflows/ci.yml Build 5 — GitHub Actions: migrations + full pipeline + pytest on every push; a gated eval-harness job on a relevant merge to main only. 2026-08-05 — migration list gained 012_chat_rounds_schema.sql (was silently missing), gained a step running the verifier's own eval harness (was never wired in at all), and every path updated for src//migrations/. 2026-08-06 — 013_allocation_items_schema.sql and a new "Build normalized allocation items" step added to both jobs' migration lists deliberately, given the previous day's bug was exactly a migration missing from one of two duplicated lists.
.gitattributes Forces consistent line endings for .sql/.yml/.md across Windows/Unix.

Docs, secrets, legacy & generated artifacts

CREDENTIALS.local.md Repo root — unredacted local companion to .env — gitignored, never shared or committed.
DEPARTMENT_MAPPING.local.md Repo root, Build 5 — the real department-code mapping used by synthesize_dataset.py, human-readable. Gitignored, same treatment as CREDENTIALS.local.md — publishing it would let anyone reverse the department masking.

data/ALLOCATION-synthesized.csv Build 5 — the project’s one official dataset (1535 rows), real temporal/quantitative fields kept, everything else synthesized. Used identically by local dev, CI, and the eval harness. Location unchanged by the reorg.
docs/sql-test-01-cheatsheet.html / .pdf This living document — every command actually typed, plus this file map. Moved from the repo root into docs/ in the 2026-08-05 reorg.
docs/ Renamed from PROJECT_DIAGRAMS/ in the reorg. One subfolder per build (BUILD_1_FLOW ... BUILD_7_FLOW) — flow diagrams (.svg/.html/.png) and handoff briefs (.html/.pdf). Build 1’s two handoff briefs live here too, recovered from Artifacts during Build 3.5.
docs/project-overview.html Build 3.5 — a seed document written to answer whole-project overview questions, so the docs-RAG corpus has something to match against.
logs/ Build 4 — gitignored directory holding tool_calls.jsonl (live, JSON Lines) and tool_calls.pre-question-id.jsonl (archived). Durable, queryable tool-call tracing — not a Postgres table, deliberately. Location unchanged by the reorg (still repo root, matching the ./logs:/app/logs volume mount).
legacy/schema_mssql.sql The SQL Server counterpart to 001_schema.sql — same appdb shape, T-SQL dialect. Moved to legacy/ in the reorg — unreferenced by any code, CI, or Docker config in this project; part of the original dual-database lab, not the seven-build agent narrative.
legacy/seed_mssql.sql SQL Server counterpart to 002_seed.sql. Same disposition as schema_mssql.sql above.
.vscode/, .pytest_cache/ Editor settings and pytest’s own cache — auto-generated, not source.

Docker

CONTAINER ORCHESTRATION

Starting the stack, and getting SQL in and out of a running container.

```
docker compose up -d --build
```

Build images if anything changed, then start every service in the background.

```
docker ps
```

List running containers — the first check when a connection unexpectedly fails.

```
docker exec -it pg_local psql -U devuser -d appdb
```

Open a **live, interactive** psql session inside the Postgres container.

```
Get-Content file.sql -Raw | docker exec -i  
pg_local psql -U devuser -d appdb
```

Run a whole .sql file inside the container **non-interactively**, PowerShell-style — < redirection doesn't exist in PowerShell, so the file gets piped in instead. (Bash/cmd would use `docker exec -i ... < file.sql`.)

```
docker exec pg_local psql -U devuser -d appdb  
-c "SELECT ... "
```

Run a single one-off SQL command non-interactively, no piped file needed. **No -it** — that flag requires a real terminal attached to stdin and fails with "cannot attach stdin to a TTY-enabled container" in non-interactive contexts (e.g. run from a script/tool rather than a live shell).

```
docker compose down -v
```

Stop everything and delete volumes too — next start re-initializes DB passwords from .env. Used deliberately, not casually.

Build 6 — COPY bakes source in at build time, with no hot-reload path at all.

Editing `agent.py/tools.py/app.py` on the host has zero effect on an already-running container, no matter what — stricter than the local Streamlit dev-server case, which at least auto-reruns `app.py` on save. A real `docker compose build + up` (recreating the container) is required every time, confirmed directly via `docker exec ... grep` against the container's own copy of a file rather than trusting the build log alone.

psql

META-COMMANDS & DDL PATTERNS

Once you're inside a psql session — both the backslash meta-commands and the recurring SQL shapes.

<code>\d table_name</code>	Describe a table — every column, its type, and any constraints. The one check that settles "did my CREATE TABLE actually work."
<code>\du role_name</code>	Describe a role — confirms it has no Superuser/Creatorole/Createdb it shouldn't.
<code>\pset null 'marker'</code>	Display NULLs as a visible marker instead of blank — otherwise a NULL and an empty string look identical.
<code>\! cls</code>	Shell out and run an OS command without leaving psql — <code>cls</code> clears the terminal screen.
<code>\q</code>	Quit psql, back to the shell prompt.
<code>CREATE SCHEMA IF NOT EXISTS name;</code>	Create a schema, skip quietly if it's already there — safe to rerun.
<code>DROP TABLE IF EXISTS name;</code>	Drop a table, skip quietly if it doesn't exist — what makes a rebuild script rerunnable from a blank database.
<code>GRANT SELECT ON ALL TABLES IN SCHEMA s TO role;</code>	Give a role read access to every table currently in a schema.
<code>ALTER DEFAULT PRIVILEGES IN SCHEMA s GRANT SELECT ON TABLES TO role;</code>	Extend that same grant automatically to any table created later — without this, new tables start private.
<code>col TYPE GENERATED ALWAYS AS (expr) STORED</code>	A column Postgres computes and locks from other columns in the same row — structurally impossible for it to drift out of sync.
<code>GRANT USAGE, CREATE ON SCHEMA s TO role;</code>	Let a role create objects inside a schema, not just read from it — how <code>appdb_agent_writer</code> got write access scoped to <code>agent_scratch</code> only, never anywhere wider.
<code>ALTER DEFAULT PRIVILEGES FOR ROLE owner IN SCHEMA s GRANT SELECT ON TABLES TO role;</code>	Extend a different role's read access to tables a specific owner role creates later in that schema — how <code>appdb_reader</code> can always see whatever <code>appdb_agent_writer</code> saves, without a re-grant each time.
<code>CREATE EXTENSION IF NOT EXISTS vector;</code>	Enable pgvector for the database — requires the <code>pgvector/pgvector</code> Postgres image, not the plain postgres one.

`ALTER DATABASE db REFRESH COLLATION VERSION;`

Clear the collation-version mismatch warning that appears after swapping to an image built against a different glibc/ICU — safe here since the data is plain ASCII text.

`col vector(384)`

pgvector's column type for a fixed-dimension embedding — 384 matches the sentence-transformers model in use, not an arbitrary choice.

`a <=> b`

Cosine distance between two vectors — smaller means more similar. What `search_summaries` orders by.

`a <-> b`

Euclidean (L2) distance between two vectors — used in this project only for the self-distance sanity check (`v <-> v` should be exactly 0).

`vector_dims(col)`

The dimensionality of a stored vector — a quick sanity check that every row actually has a real 384-dim embedding, not a malformed one.

`CREATE INDEX ... USING hnsw (col
vector_cosine_ops)`

Build an approximate-nearest-neighbor graph index matching the `<=>` operator, so similarity search can use an index scan instead of a full table scan as the corpus grows.

`EXPLAIN ANALYZE SELECT ...`

Confirms whether the planner actually used a new index (Index Scan) or fell back to Seq Scan — the only real way to verify an index is doing anything.

PowerShell

HOST-SIDE SHELL

Running Python, checking your own environment, and vetting/installing system-level tools from outside any container.

```
.\.venv\Scripts\python.exe script.py
```

Run a script with the project's own virtual-env interpreter directly — sidesteps PATH/activation issues entirely.

```
.\.venv\Scripts\python.exe -m pip install pkg
```

Install a package into that specific virtual env, not wherever PATH happens to point.

```
.\.venv\Scripts\python.exe -m pip show pkg
```

Check whether a package is installed, and exactly which version.

```
Get-ChildItem Env: | Where-Object { $_.Name -  
like "*X*" }
```

List environment variables whose name matches a pattern — e.g. catching a stray API key leaked into the shell.

```
Get-Content file
```

Print a file's contents — the go-to way to verify what actually got saved to disk.

```
claude auth status
```

Confirm this Claude Code session is on subscription login, not silently metered API billing.

```
winget show --id pkg -e
```

Pull a package's full manifest — publisher, source URL, license, install hash — before trusting or installing it.

```
winget install --id pkg -e
```

Install a package system-wide once it's been vetted, not before.

Git

CHECKPOINTING PROGRESS

Committing at the end of every phase, per the project's own working style.

`git status`

What's changed, staged, or untracked, right now.

`git add file`

Stage a specific file by name — never a blind `-A`.

`git commit -m "..."`

Record the staged snapshot with a message explaining **why**, not just what.

`git mv old new`

Rename a tracked file while git keeps its history attached to the new name.

`git log --oneline -n`

Recent commit history, one line each — the fastest way to check "where are we, really."

`git diff`

Exact unstaged changes, line by line, before they get staged.

`git remote add origin url / git push --set-upstream origin main`

Build 5 — wire up a GitHub remote for the first time and push with tracking, so plain `git push` works afterward.

`git stash push -u -m "... " / git stash pop`

Build 5 — shelve in-progress uncommitted work (including untracked files, via `-u`) to get a clean tree before a history rewrite, then restore it after.

`git commit --amend`

Build 5 — rewrite the tip commit in place. Safe only when that commit was **never pushed**; used to fix a real mistake (real data hardcoded in a script) before it ever left the machine.

`git bundle create out.bundle --all / git bundle verify out.bundle`

Build 5 — a full, portable backup of every ref in the repo in one file. Taken before every `git filter-repo` pass this build, saved outside the repo itself.

`git log --all -S"string" --oneline`

Build 5 — pickaxe search: commits where a literal string's occurrence count changed. The actual verification method used to confirm a purge worked, not assumed from the tool's own success message.

`git show <commit>:<path>`

Build 5 — print a file's exact content as of one specific commit, without checking it out — used to directly confirm a blob no longer contains something it used to.

```
git reflog expire --expire=now --all && git gc
--prune=now
```

Build 5 — force-expire the reflog and immediately garbage-collect, so an amended-away commit's old blob is actually gone locally, not just unreferenced.

```
git push --force origin main
```

Build 5 — overwrite the remote's history after a filter-repo rewrite. Only used after explicit confirmation, and only because the rewrite genuinely changed already-pushed commits.

GitHub CLI

GH · BUILD 5

Installed via winget mid-build (not present by default) — authentication and repo creation still required the user directly; everything past that ran from this session.

```
gh auth login / gh auth status
```

Authenticate gh to a GitHub account (interactive — requires the user's own browser/credentials) and confirm the current login, protocol, and token scopes.

```
gh repo create name --public --source=. --
remote=origin --push
```

Create a new GitHub repo from the current local repo and push it in one step — the intended one-liner, not usable here since gh wasn't installed yet at that point.

```
gh secret set NAME --repo owner/repo
```

Set a GitHub Actions secret, prompting for the value securely (never echoed, never logged) — how ANTHROPIC_API_KEY got configured, by the user directly.

```
gh run list --limit n
```

Recent Actions runs for the current repo — status, workflow, branch, duration.

```
gh run view <id>
```

Step-by-step breakdown of one run — which steps passed, which failed, without opening a browser.

```
gh run view <id> --log-failed
```

Full log output for only the failed step(s) — the actual diagnostic step every real CI failure this build was root-caused from.

```
gh run watch <id>
```

Streams a run's live progress until it completes — used after every push this build to confirm a fix before moving on, rather than polling manually.

Python — psycopg

TALKING TO POSTGRES

The driver behind every script in this project — agent.py, tools.py, ingest.py, profile.py.

<code>psycopg.connect(host=, dbname=, user=, password=)</code>
Open a connection to a Postgres database.
<code>with conn.cursor() as cur:</code>
Open a cursor scoped to a with block — closes itself automatically, even on error.
<code>cur.execute(sql, params)</code>
Run one SQL statement, with parameters passed safely (never string-formatted into the query).
<code>cur.executemany(sql, rows)</code>
Run the same statement once per row — how ingest.py loads 1,535 rows in one call.
<code>cur.fetchall()</code>
Pull every result row back as a Python list.
<code>cur.description</code>
Metadata about the result's columns — used to grab column names generically.
<code>conn.commit()</code>
Make written changes permanent.
<code>conn.close()</code>
Release the connection.

Python — psycopg.sql

SAFE DYNAMIC IDENTIFIERS

save_dataframe's guardrail for building a CREATE TABLE/INSERT statement where the table and column names themselves come from the model, not just the values — %s parameters alone can't protect an identifier.

```
from psycopg import sql
```

The submodule that builds SQL safely out of parts psycopg can't treat as plain data — like table and column names.

```
sql.Identifier(name)
```

Safely quotes a dynamic identifier (table/column name) — the equivalent of a parameter, but for names instead of values.

```
sql.SQL(" ... {} ... ").format( ... )
```

Builds a SQL string from literal fragments plus safely-escaped pieces like Identifier — never plain f"..." string interpolation.

```
sql.SQL(", ").join(parts)
```

Joins a list of Identifier/SQL fragments with a separator — how a variable-length column list gets assembled.

```
sql.Placeholder()
```

A %s value placeholder built the same composable way, for when the number of columns isn't known until runtime.

Python — pandas

PROFILING RAW DATA

Everything profile_raw_data.py used to understand raw.allocations before designing the clean schema. (Renamed from profile.py in Build 3 — it was silently shadowing Python's stdlib profile module.)

```
pd.DataFrame(rows, columns=cols)
```

Build a DataFrame straight from plain rows + column names — no SQLAlchemy required.

```
df["col"].unique()
```

Every distinct value in a column, once each — printed with repr() to expose hidden whitespace.

```
df["col"].value_counts(dropna=False)
```

Count of each distinct value, including blanks/NaN.

```
pd.to_datetime(s, format=, errors="coerce")
```

Parse a column of date strings against an exact format; anything that doesn't match becomes NaT instead of crashing the run.

```
series.dt.total_seconds()
```

Convert a time difference into a plain number of seconds.

```
series.str.fullmatch(pattern)
```

Test every value in a column against a regex at once — used to confirm two columns were digit-only.

Python — pandera

DATA-QUALITY SCHEMAS

test_data_quality.py uses this to declare what "correct" looks like for clean.allocations, as data instead of ad hoc checks.

```
import pandera.pandas as pa
```

The explicit pandas-backend import for modern pandera versions.

```
pa.DataFrameSchema({ ... })
```

Declare an entire DataFrame's expected shape — one entry per column — as data, not scattered asserts.

```
pa.Column(type, checks, nullable=)
```

One column's expected type, whether NULLs are allowed, and any constraints.

```
pa.Check.ge(0)
```

A reusable constraint — here, "greater than or equal to 0" — attached to a Column.

```
schema.validate(df)
```

Check a real DataFrame against the schema; raises a real error the moment something doesn't match.

pytest

RUNNING THE TEST SUITE

First automated test suite in the project — codifies checks that were previously done by hand.

```
.\.venv\Scripts\python.exe -m pytest -v
```

Discover and run every test_* function in the project, printing each one's name and pass/fail individually.

```
def test_something():
```

Any function named test_... in a test_*.py file is auto-discovered — no registration needed.

```
assert condition
```

Plain Python assert — pytest reports exactly which assertion failed and the actual vs. expected values.

```
with pytest.raises(ExceptionType):
```

Asserts a block **must** raise a specific exception — used to prove a role boundary actually blocks a write, not just that it "seems fine."

```
def test_x(monkeypatch):
    monkeypatch.setattr(mod, "NAME", val)
```

Temporarily overrides a module-level constant for one test only, auto-restored after — how a cap like MAX_SCRATCH_TABLES gets tested at its boundary without creating 50 real tables.

pgvector & embeddings

SENTENCE-TRANSFORMERS · PGVECTOR.PSYCOPG

Turning free text into vectors and getting Postgres to store/compare them —
embed_summaries.py, search_summaries.py, tools.py's
search_summaries.

```
from sentence_transformers import  
SentenceTransformer
```

A local, free embedding model library — no API key or per-call cost, unlike Voyage AI/OpenAI embeddings.

```
SentenceTransformer("all-MiniLM-L6-v2")
```

Loads the specific 384-dim model used throughout this project — first call downloads and caches it locally.

```
model.encode(text_or_texts,  
show_progress_bar=True)
```

Turns one string or a list of strings into embedding vector(s) — a progress bar matters when embedding 1,500+ rows at once.

```
from pgvector.psycopg import register_vector
```

Teaches a psycopg connection how to send/receive pgvector's vector type directly as numpy-like arrays, instead of raw strings.

```
register_vector(conn)
```

Applies that registration to a specific connection — needed once per connection, before any vector query runs.

```
cur.execute(" ... WHERE col ⇔ %s < %s ... ",  
(embedding, threshold))
```

Passes a computed embedding straight in as a query parameter — the distance operator and the threshold cutoff both happen inside the SQL, not in Python after fetching everything.

Python — BeautifulSoup

HTML PARSING & CHUNK EXTRACTION

Turning this project's own handoff briefs/cheat sheet into embeddable chunks
— `extract_doc_chunks.py`, Build 3.5.

```
from bs4 import BeautifulSoup
```

The HTML parsing library — reads real markup instead of treating a doc as one flat blob of text.

```
BeautifulSoup(html_text, "html.parser")
```

Parses a string of HTML into a navigable tree, using Python's built-in parser (no extra system dependency like `lxml`).

```
soup.select(".callout")
```

CSS-selector search — returns every element matching a class/tag, the same selector syntax as the CSS already written for these docs.

```
soup.select_one(".fn")
```

Like `select`, but returns just the first match (or `None`) — used when a chunk has at most one label element.

```
element.get_text(" ", strip=True)
```

Pulls plain text out of an element and its children, joining pieces with a space and trimming whitespace — turns nested markup into one clean string to embed.

Python — standard library

CSV · OS · DOTENV

No extra install required — these ship with Python itself, or python-dotenv.

`csv.DictReader(f)`

Read a CSV where each row comes back as a `{column: value}` dict — chosen over pandas for ingestion specifically so nothing gets silently type-coerced.

`open(path, newline="", encoding="utf-8-sig")`

Open a file safely for CSV reading, transparently stripping a leading BOM if present.

`os.environ["KEY"]`

Read a required env var — raises immediately if it's missing, rather than continuing with a silent gap.

`os.environ.get("KEY", default)`

Read an optional env var, falling back to a default (e.g. `POSTGRES_HOST` defaulting to `127.0.0.1` outside Docker).

`load_dotenv(encoding="utf-8-sig")`

Load `.env` into the process environment — BOM-safe, matching how Windows tools often save that file.

`json.dumps(entry, default=str)`

Serialize a dict to a JSON string, falling back to `str()` for anything the default encoder can't handle (e.g. `datetime`) — how `logging_utils.py` keeps every logged line valid JSON regardless of what a tool result contains.

`open(path, "a") as f:`
`f.write(json.dumps(entry) + "\n")`

The JSON Lines pattern — one independent JSON object per line, appended, never rewriting the whole file. Read back with `json.loads(line)` per line, not one big `json.load()`.

`uuid.uuid4().hex[:12]`

A short random ID with no coordination needed across processes — how each `ask_agent()` call gets a distinct `question_id`. Not a counter: a shared counter would itself need synchronization once calls can run concurrently.

`time.perf_counter()`

A monotonic, high-resolution clock for measuring elapsed time — call before and after, subtract, multiply by 1000 for milliseconds. Not `time.time()`, which can jump if the system clock adjusts.

`sys.argv[1] if len(sys.argv) > 1 else default`

Read an optional CLI argument, falling back to a default — how `run_eval.py` picks which eval-case module to run.

`importlib.import_module(name)`

Import a module by its name as a string, decided at runtime — lets `run_eval.py` load `eval_cases` or `eval_cases_round2` (or any future round) with no `if/elif` import chain.

`from collections import Counter;`
`Counter(items).most_common()`

Count occurrences of each distinct value and list them ranked most-to-least frequent — `metrics_rollup.py`'s tool-usage frequency.

```
n = len(open(path).readlines()); ... ;  
f.readlines()[n:]
```

Checkpoint-based tailing — count existing lines before an action, then re-read only the lines appended after. How `run_eval.py` isolates one `ask_agent()` call's new log entries without `ask_agent()` needing to return that data itself.

Streamlit

BUILD 6 · APP.PY

Everything `app.py` uses to turn `ask_agent()` into a real chat interface — session state, chat primitives, and the Phase 4 tool-call expander.

```
streamlit run src/app.py --
server.address=0.0.0.0
```

Starts the dev server. The address flag is only needed inside Docker — Streamlit defaults to binding `localhost` only, unreachable through a container's port mapping otherwise; not needed for a plain local run.

`st.session_state`

A dict-like store that survives Streamlit's full-script rerun on every interaction — a plain Python variable would reset to its initial value every time. Holds live Python objects directly; no JSON serialization needed within one session.

```
if "key" not in st.session_state:
    st.session_state.key = default
```

The standard init-once guard — runs on every rerun, but only actually sets the default the first time.

`st.chat_message(role)`

A styled message bubble for a given role (`"user"/"assistant"`) — used as a `with` block, contents render inside it.

`st.chat_input(placeholder)`

The bottom-anchored chat text box — returns the typed text once submitted, otherwise `None`.

`st.spinner(" ... ")`

Shows a loading indicator for the duration of a `with` block — wraps the `ask_agent()` call so the wait for a real API round-trip is visibly acknowledged.

`st.error(msg)`

A visually distinct red message box — used to surface `ask_agent()`'s error field separately from a normal answer, not swallowed.

`st.expander(label)`

A collapsible section, closed by default — the `Tools used (N)` block per answer, keeping tool-call detail available without it dominating the page.

`st.code(text, language="json")`

A syntax-highlighted, monospace code block — used for each tool call's formatted input.

`st.caption(text)`

Small, muted secondary text — the `PROCESS_ID` line under the title, and each tool call's latency.

Build 6 Phase 3.5 — `st.expander` cannot nest inside another `st.expander`. Hit while building the FAQ-style collapsed-round view (each older round wraps its answer in an expander labeled with the question) — a round's `Tools used` detail (itself normally an expander, Phase 4) has to render as plain inline text/captions instead whenever it's inside an already-collapsed round, and only get the real widget when that round is the one currently shown expanded.

Anthropic SDK

BUILD 7 · VERIFY_ANSWER.PY

One genuinely new API pattern this build — forcing a specific tool call instead of letting the model choose freely.

```
tool_choice={"type": "tool", "name":  
"report_verdict"}
```

Forces the model to call one specific tool, every time — no `stop_reason == "end_turn"` text-only path exists. How the verifier's output is guaranteed structured (`{supported, reason}`) instead of prose that would need parsing.

Claude's Commands

DIAGRAMS · VERIFICATION · PUBLISHING

A separate running list from everything above — not what *you've* typed, but the commands Claude actually runs behind the scenes while building diagrams, verifying renders, and publishing docs for this project. Appended to as new patterns come up, same as every other section.

2026-08-06 — full incident audit: the same day's token-count feature request led to a verifier false positive, a 36.5s latency regression, a single 177,090-token wrong answer, four prompt-fix iterations, and a root-cause data normalization — documented question-by-question, with real timestamps, in [2026-08-06-token-burn-incident-audit.html](#).

<pre>msedge --headless --disable-gpu --window-size=W,H --screenshot=out.png file:///path.html</pre>	Renders an HTML file exactly as a browser would and saves it as a PNG — the first of two required checks before any diagram or doc is considered verified.
<pre>msedge --headless --disable-gpu --print-to-pdf=out.pdf --print-to-pdf-no-header --no-pdf-header-footer file:///path.html</pre>	Renders the same page through Chromium's separate print pipeline — a genuinely different code path that has caught real bugs (references silently failing to render) the screenshot path alone missed.
<pre>pdftoppm -png -r 100 file.pdf prefix</pre>	Rasterizes every page of a generated PDF into numbered PNGs, so each page can actually be looked at instead of trusting file size alone.
<pre>pdfinfo file.pdf grep -i pages</pre>	Quick page-count check before rasterizing — confirms whether a doc paginated the way it was expected to.
<pre>git commit -m "\$(cat <<'EOF' ... EOF)"</pre>	Heredoc commit-message convention — explains why a change happened in the body, and avoids hand-escaping quotes/newlines. No Co-Authored-By trailer as of 2026-08-04 — the full history was rewritten via <code>git filter-repo</code> to strip it from all commits ahead of public/recruiter-facing use, and it's no longer appended going forward.
<pre>Image.open(path).convert("RGB").getpixel((x, y))</pre>	Samples an exact on-screen pixel's RGB value with Pillow — confirms a color rendered exactly as CSS specified, rather than trusting a visual impression at small/compressed scale. Caught a false alarm (anti-aliasing fringe mistaken for a real color bug) during this cheat sheet's own verification.
<pre>WebFetch(url="claude.ai/code/artifact/<uuid>", ...)</pre>	For Artifacts the user owns, this returns full raw HTML rather than the usual markdown conversion — used to recover Build 1's two handoff briefs, which had only ever been published as Artifacts, never saved as local files.

```
cygpath -m "$(pwd)/file.html"
```

Converts a Git Bash path to a Windows-style forward-slash path (C:/...) — necessary before building a file:/// URL for headless Edge, since Git Bash's own /c/... path style isn't a valid Windows path and silently fails to load.

```
git log --oneline <hash>^..<hash>
```

The exact commit range between two specific commits (the leading ^ makes it inclusive of the first one) — used to pin down a build's full commit range for its handoff brief, rather than -n's "last N commits."

```
pdftotext file.pdf -
```

Build 5 — extracts a PDF's actual text to stdout. Used specifically because a plain grep on a PDF's raw bytes can silently miss encoded/compressed text — the real check after redacting something from a rendered document.

```
gh run view <id> --log-failed
```

2026-08-05 — a real CI failure email led here: 23 commits (all of Build 6 Phase 2 onward) had been sitting local-only, never pushed, so CI had never actually run against any of it until a history rewrite finally pushed everything. Root cause was ci.yml's migration list silently missing 012_chat_rounds_schema.sql — same bug class as extract_doc_chunks.py's old hand-maintained SOURCE_FILES list. Fixed, then watched a fresh gh run watch go green before trusting it.

```
git mv old/path new/path
```

2026-08-05 — used for the full repo reorg (src/, migrations/, tests/, docs/, legacy/), preserving each file's git history as a rename rather than a delete+add. The same CI failure that surfaced the migration-list bug prompted a full audit, which also found .env.example documenting 4 of the 10 required env vars and no README.md at all — both fixed in the same pass.

```
grep -c '"tool_name"' logs/tool_calls.jsonl |  
sort by result size
```

2026-08-06 — a new token-usage counter (in the console/UI, not a new log field on its own) surfaced a real question that cost 70,007 tokens. Rather than guess, grepped every tool result's serialized size across the whole log: 31 past results exceeded 5,000 characters, one hit 936,551 — almost certainly the exact cause of an already-documented Build 6 outage ("prompt too long: 448,866 tokens"). The proximate cause was never row count (already capped at 200) — it was individual field size (a pipe-delimited summary value, or a vector (384) embedding column pulled in by SELECT *), which nothing capped at all. Confirms the general lesson: when a metric jumps, pull the real distribution across history before proposing a fix — one data point can mislead, a full sweep usually can't.

```
for i in 1 2 3; do python run_eval.py; done
```

Re-running an eval case multiple times in a row before trusting a pass or a fail — a substring check on LLM-generated prose is not deterministic by construction, so one run of either result proves nothing on its own. Caught two real, different classes of flakiness this way 2026-08-06: a case whose correct answer sometimes phrased around a required literal identifier (fixed by making the question more directive, not by declaring the check broken), and a case whose correct answer legitimately reaches the same conclusion via more than one tool (fixed by allowing expected_tool to be a list). A third run in this same loop surfaced an unrelated real bug (a UnicodeEncodeError crash) that a single run would never have shown at all.

```
docker exec <container> python -c  
"... search_summaries('camera') ..." (fresh  
process, no Streamlit)
```

2026-08-06 — isolated a 36.5s latency spike from Streamlit itself by running the identical embedding call in a plain, non-Streamlit process inside the same container. Still 36.2s — directly disproved the first hypothesis (Streamlit's file watcher) rather than accepting a plausible-looking correlation. A second fresh process, run after the first had already warmed the model cache, measured 2.4s — pointed straight at the real cause (no persistent cache, full network re-download on every fresh container).

```
Python: read every summary value directly,  
split on '|' then ' - ', tally with  
collections.Counter
```

2026-08-06 — audited the full universe of distinct equipment item names (55, across 1,535 allocations) by parsing every row in plain Python rather than trusting any single SQL ILIKE guess. Found the "CAMERA CASE contains CAMERA" trap recurs across at least 7 other product families, plus a second bug (real cameras inconsistently named "... BODY" vs "... CAMERA"). This audit became the classification source for `src/build_allocation_items.py`.

Headless Edge sometimes needs a command run twice. The first invocation can exit with code 2 and no output file at all; an identical second call then succeeds. Observed repeatedly across both screenshot and print-to-pdf renders — not project-specific, a Windows/Chromium quirk. Always confirm the output file actually exists (and, for PDFs, that `pdftinfo` reports the expected page count) after a render — never assume the first call worked.

Living document — appended at the end of each build, not written once and frozen. Reflects hands-on usage through Build 7, complete end to end — a second, cheaper agent now checks every real tool-backed answer against the actual evidence before it's returned, surfaced as a green/red badge in the Streamlit UI and proven (via a dedicated eval harness) to actually catch a bad answer, not just rubber-stamp everything. Not exhaustive documentation for any of these tools, just what's actually been typed in this project so far.